

npuloop: 가상 NPU 비용 모델과 비트 정확 정수 엔진으로 달는 모델 압축 루프

기술 보고서 · 2026-09-11 · 저장소 github.com/sokldjs554/npuloop

초록

NPU 회사의 모델 팀은 고객이 보낸 체크포인트에 대해 "우리 칩에서 돌기는 하는가, 얼마나 걸리는가, 무엇을 바꿔야 하는가"를 답해야 한다. 이 보고서는 그 세 질문을 숫자로 답하는 오픈소스 툴킷 npuloop의 설계와 실험을 정리한다. torch.fx 그래프 IR 위에 (1) SCALE-Sim으로 검증한 weight-stationary systolic-array 비용 모델, (2) NPU 준비도 lint, (3) 비트 단위로 일치하는 두 정수 엔진(NumPy·C++)과 파이썬 없는 독립 실행기, (4) 비용 모델을 루프 안에서 호출하는 그래프 기반 구조적 프루닝을 엮었다. CIFAR-10 5개 모델(ResNet-20 ReLU/SiLU, MobileNetV2-0.5, Inception-32, ViT-128/6)과 Imagenette 128×128에서 fake-quant와 정수 실행의 불일치를 노드별로 국소(teacher-forced)/전파로 분리해 재고, 정수 구현 세부(반올림·곱셈기 비트·누산기·바이어스 폭)의 정확도 비용을 ablation했으며, TensorFlow Lite reference 커널과 1,000장 × 모든 텐서에서 비트 일치함을 보였다. 주요 결과: fake-quant는 정확도 예측기로 충분하지만(주 스킴에서 $|\Delta| \leq 0.20\%$) 텐서 단위로는 출력 코드의 17~84%가 다르고, transformer에서는 LayerNorm이 국소 불일치의 대부분을 차지하며, TFLite reference 커널은 conv/pool과 fully-connected가 서로 다른 반올림을 쓴다.

1. 문제

INT8 NPU에 모델을 올리는 작업은 세 층위의 불일치를 다룬다. (a) 학습 프레임워크의 fake-quant와 실제 정수 실행의 차이, (b) FLOPs와 사이클의 차이(배열 정렬·채움/비움·메모리), (c) 같은 "INT8"이라도 런타임마다 다른 requantization 구현. 공개 도구는 이 셋을 따로 다룬다: MQBench·PPQ는 (a)를 백엔드 정확도 격차로, SCALE-Sim·Timeloop는 (b)를 오프라인 시뮬레이션으로, TFLite·gemmlowp는 (c)를 코드로만. npuloop의 기여는 셋을 한 그래프 IR 위에 놓고, 결정(프루닝·활성함수 교체·PTQ/QAT)이 실제 정수 결과와 사이클로 되돌아오게 한 것이다.

2. 방법

IR. torch.fx 로 추적한 그래프에서 BN을 접고 shape를 전파한 StaticGraph.conv/linear(토큰 단위 포함)/add/mul/concat/matmul/softmax/ layernorm/transpose/reshape/pool/활성함수를 지원하고 나머지는 즉시 실패시킨다.

비용 모델. weight-stationary systolic array에서 GEMM 한 타일의 사이클을 $M + 2R + C - 2$ (M 출력 픽셀, $R \times C$ 배열)로 두고 $[K/R] \cdot [N/C]$ 타일을 합산, 멀티코어 M/N 분할, depthwise 엔진, LUT/융합/호스트 폴백 활성함수, DRAM roofline을 더한다. SCALE-Sim v3(사이클 정확, WS)와 4개 모델 × 3개 배열 크기에서 합계 0.1% 이내로 일치한다(표 6). 에너지는 MAC·SRAM·DRAM·벡터·호스트 항목의 pJ 상수(Horowitz 2014 자릿수)로 같은 양에서 추정하며 simulated 라벨을 단다.

정수 엔진. 캘리브레이션된 fake-quant 그래프를 int8 가중치·int32 바이어스·Q31 곱셈기+시프트의 IntGraph로 export하고, gemmlowp/TFLite의 MultiplyByQuantizedMultiplier 의미론으로 실행한다. NumPy 구현과 C++ 커널(ctypes)은 모든 중간 텐서에서 비트 일치를 테스트로 강제하고, .npuloop 파일(JSON 헤더 + raw 정수 배열)을 파이썬 없이 실행하는 C++ 러너가 같은 답을 낸다. RequantConfig로 반올림 모드·곱셈기 비트·누산기/바이어스 폭을 바꿔 "짜구려 구현"을 흉내 낼 수 있고, 노드별 반올림 오버라이드도 가능하다.

검증. 같은 이미지에서 fake-quant 코드와 정수 코드를 노드별로 비교하되, 두 가지로 켜다: **전파 불일치**(끝까지 정수로 실행)와 **국소 불일치**(각 노드에 fake-quant의 입력 코드를 teacher-forcing으로 넣고 그 노드의 출력만 비교). 국소 값이 어느 연산이 차이를 *만들어내는지*를, 전파 값이 그 차이가 어디까지 *번지는지*를 보여 준다.

프루닝. 그래프에서 conv/linear 출력 채널을 활성함수·depthwise conv를 지나 소비자까지 따라가 그룹을 찾는다(residual add·pool·matmul·LayerNorm에 닿으면 제외, concat을 지나면 소비자의 입력 슬라이스를 함께 자름). uniform·aligned·cost-greedy(사이클 절감/중요도 손실이 큰 그룹부터, 비용 모델을 매 단계 호출) 세 전략.

3. 실험 설정

CIFAR-10 공식 학습 50,000장을 고정 시드로 학습 45,000 / 검증 5,000(클래스별 500)으로 나누고 검증 정확도로 체크포인트를 고르며 test 10,000장은 마지막에 한 번 평가한다. 모델 5개: ResNet-20 ReLU, ResNet-20 SiLU, MobileNetV2-0.5 ReLU6, 고객 A · ViT-128/6, 고객 B · Inception-32. 가상 NPU 프리셋 4개(tiny-1tops, edge-10tops, pcie-80tops, edge-10tops-strict). 캘리브레이션은 학습 분할에서 512장. 모든 학습·실험은 CPU 4코어에서 seed 0 한 번으로, 0.2%p 이하의 정확도 차이는 잡음이다(10k 표준오차 $\approx$ 0.3%p).

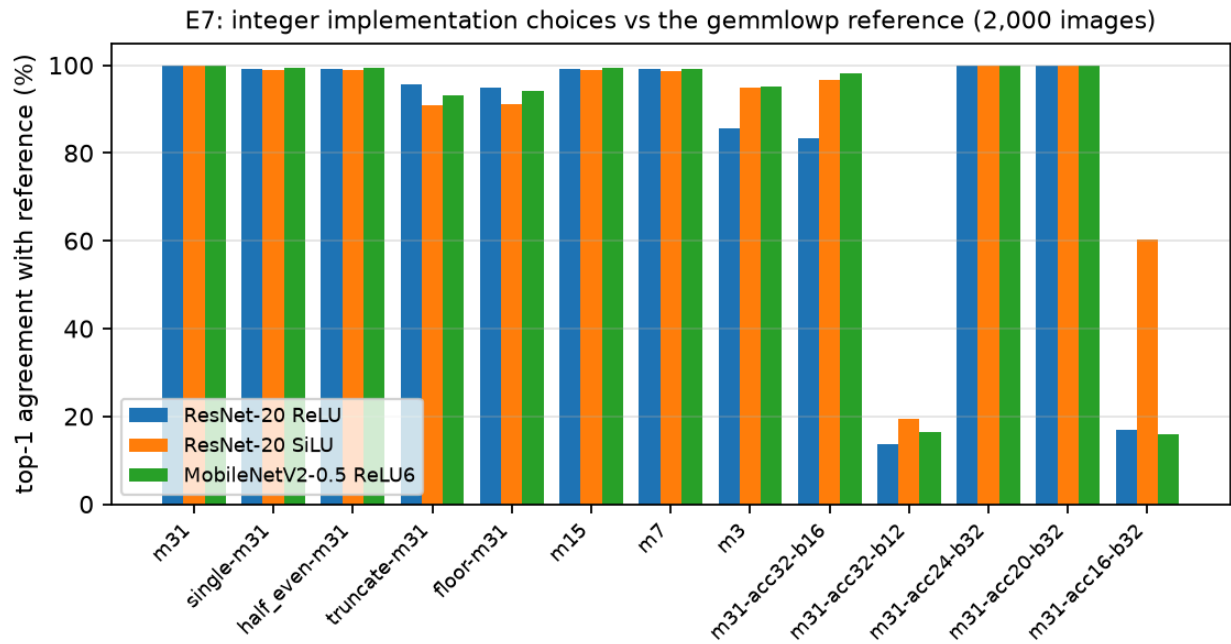
4. 결과

4.1 fake-quant는 정확도를 맞히지만 텐서는 맞히지 않는다 (E2)

모델	스킵	이미지	fake-quant	정수 엔진	차이	top-1 일치	출력 코드 불일치
ResNet-20 ReLU	npu-default	10,000	89.54%	89.55%	+0.01%p	98.8%	50%
ResNet-20 ReLU	per-tensor	10,000	89.61%	89.67%	+0.06%p	99.6%	51%
ResNet-20 SiLU	npu-default	10,000	90.35%	90.26%	-0.09%p	98.4%	60%
ResNet-20 SiLU	per-tensor	10,000	90.17%	90.26%	+0.09%p	99.2%	63%
고객 A · ViT-128/6	npu-default	10,000	80.87%	80.97%	+0.10%p	98.0%	71%
고객 A · ViT-128/6	per-tensor	10,000	80.77%	80.97%	+0.20%p	98.4%	72%
고객 B · Inception-32	npu-default	10,000	89.55%	89.56%	+0.01%p	100.0%	27%
고객 B · Inception-32	per-tensor	10,000	89.52%	89.47%	-0.05%p	99.2%	31%
MobileNetV2-0.5 ReLU6	npu-default	10,000	90.30%	90.29%	-0.01%p	99.6%	50%
MobileNetV2-0.5 ReLU6	per-tensor	10,000	90.23%	90.27%	+0.04%p	100.0%	52%

주 스킵에서 정확도 차이는 최대 0.20%p, 이미지 단위 top-1 일치는 95.2~100.0%다. 그러나 출력 코드는 17~84%가 다르다. 각 conv의 국소 불일치는 0.03~0.2%(± 1 LSB)뿐이며 나머지는 나비효과다.

4.2 정수 구현 세부의 비용 (E7)

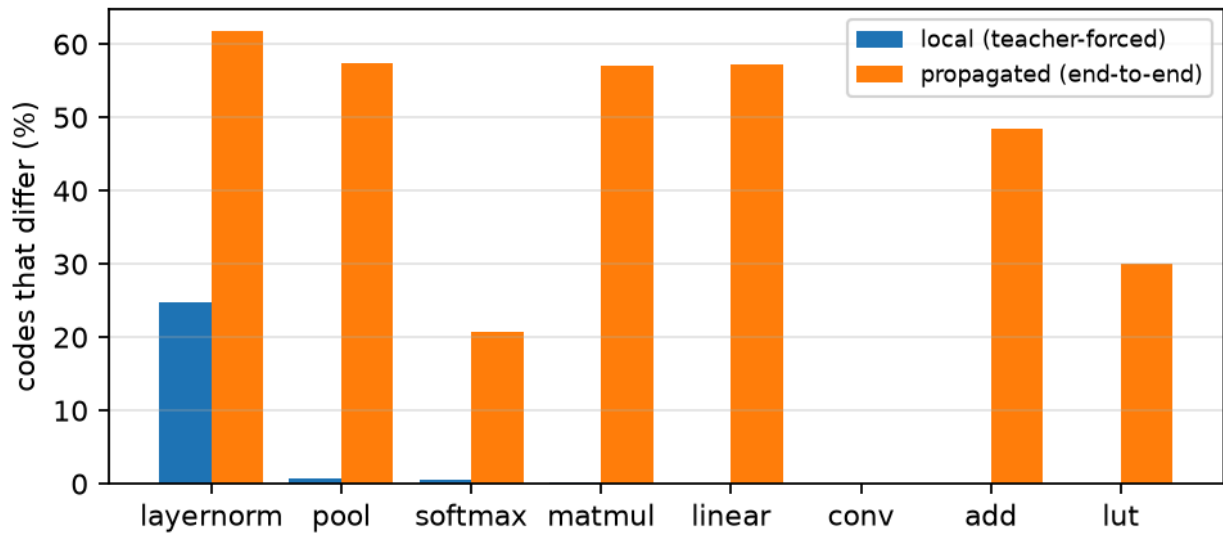


RequantConfig (ResNet-20 ReLU)	정확도	기준과 top-1 일치	포화 횟수
tfllite-m31-acc32-b32	90.10%	100.0%	0
single-m31-acc32-b32	90.10%	99.2%	0
half_even-m31-acc32-b32	90.20%	99.2%	0
truncate-m31-acc32-b32	89.45%	95.6%	0
floor-m31-acc32-b32	89.10%	94.8%	0
tfllite-m15-acc32-b32	90.20%	99.2%	0
tfllite-m7-acc32-b32	90.35%	99.2%	0
tfllite-m3-acc32-b32	81.90%	85.7%	0
tfllite-m31-acc32-b16	79.55%	83.3%	0
tfllite-m31-acc32-b12	13.45%	13.6%	0
tfllite-m31-acc24-b32	90.10%	100.0%	0
tfllite-m31-acc20-b32	90.10%	100.0%	0
tfllite-m31-acc16-b32	16.80%	17.0%	11,152,196

반올림 모드(single/half-even)는 무관하지만 truncate/floor는 1~3%p를 잃고, 3비트 곱셈기는 ResNet-20 ReLU에서 -8%p, 바이어스 int16은 최대 -10%p, 누산기 int16은 모델을 무너뜨린다(1,115만 회 포화). 이 순서는 세 모델에서 같다.

4.3 transformer에서 차이를 만드는 곳은 LayerNorm이다 (E9)

E9: fake-quant vs integer engine on the ViT, per op kind (64 images)

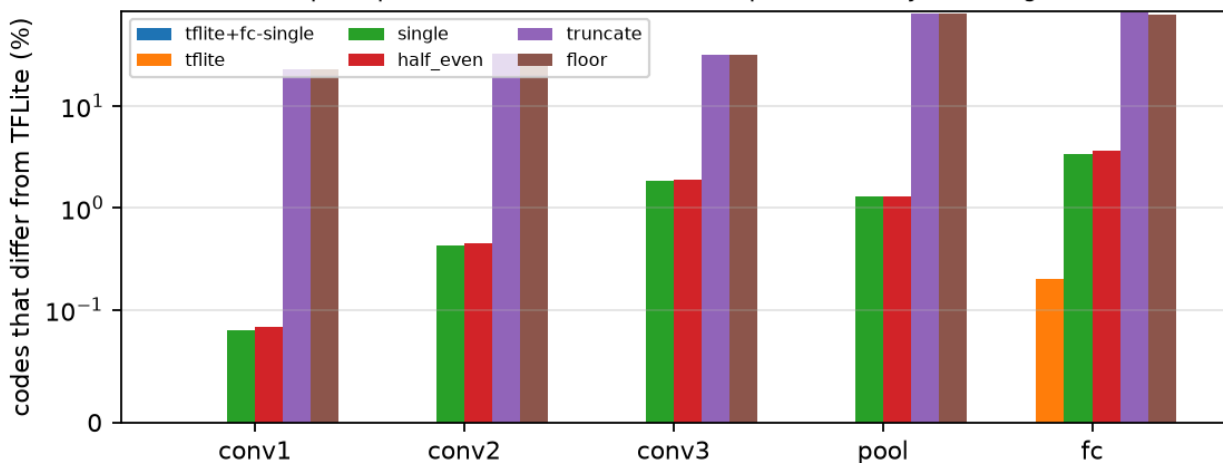


op	노드 수	국소 불일치 평균	최대	전파
layernorm	13	24.74%	26.11%	61.8%
pool	1	0.76%	0.76%	57.3%
softmax	6	0.49%	0.81%	20.8%
matmul	12	0.27%	0.43%	57.1%
linear	37	0.08%	0.16%	57.3%
conv	1	0.04%	0.04%	0.0%

ViT-128/6의 최종 출력 코드 불일치는 74%(CNN 30~50%)이고 top-1 일치는 96.9%다. 국소 불일치는 LayerNorm이 압도적이다: 정수 LayerNorm은 int64 합과 정확한 정수 제곱근으로 정규화하고 fake-quant는 float32로 계산한 뒤 격자에 올리므로 네 개 중 하나꼴로 반올림 경계 반대편에 떨어진다(모두 ± 1 LSB).

4.4 실제 런타임과의 비트 일치 (E11)

E11: npuloop vs TFLite reference kernels, per tensor, by rounding mode



반올림 conv.pool / fc	출력이 다른 이미지	다른 원소	conv1 / conv2 / conv3 / pool / fc 국소 불일치
tflite / single	0/1000	0/10000	0.00% / 0.00% / 0.00% / 0.00% / 0.00%
tflite / tflite	20/1000	20/10000	0.00% / 0.00% / 0.00% / 0.00% / 0.20%
single / single	155/1000	337/10000	0.08% / 0.43% / 1.86% / 1.29% / 3.37%
half_even / half_even	182/1000	364/10000	0.09% / 0.44% / 1.91% / 1.30% / 3.64%

반올림 conv·pool / fc	출력이 다른 이미지	다른 원소	conv1 / conv2 / conv3 / pool / fc 국소 불일치
truncate / truncate	1000/1000	8203/10000	23.10% / 32.78% / 32.13% / 80.23% / 82.03%
floor / floor	1000/1000	7968/10000	23.10% / 32.78% / 32.13% / 80.23% / 79.68%

TFLite full-integer 모델의 파라미터를 그대로 읽어 만든 정수 그래프는 conv·pool을 gemmlowp 이중 반올림, fully-connected를 단일 반올림으로 실행할 때 TFLite reference 커널과 1,000장 × 모든 텐서에서 0개 불일치다. 그래프 전체를 한 모드로 두면 fc에서 0.2%가 ±1 LSB 어긋난다. TFLite 자신의 XNNPACK 델리게이트는 reference 커널과 155/1000장에서 다르다.

4.5 실측과 모델을 같은 표에 (E10)

모델	NumPy ms/장	C++ ms/장	C++/NumPy	두 엔진 출력 동일	모델 사이클 (edge-10tops)
ResNet-20 ReLU	20.4	10.5	1.9×	2,000/2,000	34,417
ResNet-20 SiLU	25.9	10.8	2.4×	2,000/2,000	34,785
MobileNetV2-0.5 ReLU6	66.5	20.5	3.3×	2,000/2,000	87,056
고객 A · ViT-128/6	70.8	22.5	3.2×	2,000/2,000	52,246
고객 B · Inception-32	18.9	10.7	1.8×	2,000/2,000	31,224

이 저장소가 실측할 수 있는 시간은 호스트 CPU의 검증 엔진뿐이다. 처음 잔 C++ 엔진은 순진한 7중 루프라 NumPy(내부는 torch conv2d)보다 3배 느렸고, im2col + int32 GEMM(4탭 블록, $K \cdot \max|x-zp| \cdot 128 < 2^{31}$ 일 때만 int32 누산)으로 고쳐 2~3배 빠르게 만들었다.

4.6 비용 모델 검증 (E8)

모델	배열	SCALE-Sim cycles	npuloop cycles	비율	최악 레이어 오차
ResNet-20 ReLU	32×32	84,276.0	84,298	1.0003	0.5%
ResNet-20 ReLU	64×64	49,123.0	49,145	1.0004	0.5%
ResNet-20 ReLU	16×16	208,486.0	208,508	1.0001	0.5%
MobileNetV2-0.5 ReLU6	32×32	108,030.0	108,066	1.0003	0.2%
MobileNetV2-0.5 ReLU6	64×64	70,074.0	70,110	1.0005	0.2%
MobileNetV2-0.5 ReLU6	16×16	217,226.0	217,262	1.0002	0.1%
고객 B · Inception-32	32×32	96,108.0	96,124	1.0002	0.2%
고객 B · Inception-32	64×64	43,720.0	43,736	1.0004	0.2%
고객 B · Inception-32	16×16	289,122.0	289,138	1.0001	0.1%
고객 A · ViT-128/6	32×32	122,950.0	122,988	1.0003	0.3%
고객 A · ViT-128/6	64×64	49,620.0	49,658	1.0008	0.3%
고객 A · ViT-128/6	16×16	340,898.0	340,936	1.0001	0.3%

검증 범위는 dense conv·linear의 systolic 연산 사이클(단일 코어, 메모리 스톨 없음)이다. 처음 모델(M + R + C)은 10~13% 낙관적이었고 가중치 타일 적재 R 사이클이 빠져 있었다.

4.7 두 번째 데이터셋 (E12)

Imagenette 128×128, ResNet-20(stem stride 2): val 84.70% → test 83.97%, 163M MACs; PTQ npu-default fake 84.08% / 정수 84.03%, per-tensor fake 83.97% / 정수 84.05%, pow2 fake 84.08% / 정수 83.95%

5. 한계

CIFAR-10 규모, seed 1개, 가상 NPU(실제 컴파일러의 fusion·타일링·메모리 스케줄링 없음), 에너지는 자릿수 추정, TFLite 교차 검증은 conv·pool·fc에 한정, ImageNet 사전학습 체크포인트 미사용(호스트 차단). 체크포인트를 바꾸면 결론 일부가 흔들렸다: E3의 레이어 수준 상관은 재현되지 않았고 E7의 3비트 곱셈기 손실은 체크포인트에 따라 -1.3에서 -8.2%p까지 달랐다.

6. 결론

fake-quant 정확도는 믿어도 되지만 텐서는 믿으면 안 되고, "INT8"은 런타임마다 다른 반올림을 뜻하며, 프루닝의 값은 사이클로 환산했을 때만 드러난다. 이 세 문장을 재현 가능한 숫자로 만든 것이 npuloop이다.

참고문헌

Jacob et al. 2018 (CVPR) 정수 산술 추론; gemmlowp/TFLite MultiplyByQuantizedMultiplier;
Samajdar et al. 2020 SCALE-Sim; Nagel et al. 2019 DFQ; Liu et al. 2017 Network Slimming; Li et al. 2021 MQBench; Kim et al. 2021 I-BERT; Horowitz 2014 (ISSCC) 에너지 수치.